

APUNTE ACTIVIDAD 3

Lo que se traduce mal y las trampas invisibles

Índice

4 Lo que se traduce mal

Streams, sobrecarga y los reflejos que fallan

- 4.1 · Streams → comprehensions
- 4.2 · Sobrecarga de constructores
- 4.3 · El punto y coma aplastado
- 4.4 · Los tipos

5 Trampas que Java no te enseñó a ver

Cuatro errores que en Java simplemente no pueden pasar

4 Lo que se traduce mal

Streams, sobrecarga y los reflejos que fallan

Este capítulo reúne las trampas que aparecen cuando traducís Java a Python «por reflejo»: casos donde el resultado funciona pero delata el origen, o donde directamente ni siquiera es Python. Tres de las cuatro fueron detectadas en material real de la cátedra -la sobrecarga es la excepción: es una trampa general del pasaje-, lo cual conviene decir sin dramatismo: son los errores que comete alguien que sabe lo que está haciendo y traduce en piloto automático. Por eso mismo valen como material didáctico.

4.1 · Streams → comprehensions

El pipeline de streams de Java y la comprehension de Python resuelven el mismo problema - transformar y filtrar colecciones sin escribir un for con acumulador- y llegaron a soluciones parecidas por caminos distintos. Python tuvo comprehensions desde el 2000; Java tuvo streams desde el 8, en 2014. La equivalencia es conceptual y casi siempre directa, pero la sintaxis no se parece en nada, y ahí es donde el reflejo falla.

Este es un caso real, sobreviviendo íntegro dentro de un manual de Python:

JAVA - INCORRECTO EN CONTEXTO PYTHON

```
Map<Comisario,Long> conteo = eventos.stream()
.flatMap(e -> e.getComisarios().stream())
.collect(groupingBy(c -> c, counting()));
```

PYTHON IDIOMÁTICO

```
from collections import Counter
conteo = Counter(c for e in eventos for c in e.comisarios)
maximo = max(conteo.values())
mas_frecuentes = [c for c, n in conteo.items() if n == maximo]
```

Vale la pena desarmar la conversión porque el patrón se repite. El flatMap -aplanar una colección de colecciones- se vuelve el **doblo for dentro de la comprehension**: for e in eventos for c in e.comisarios. Se lee en el mismo orden en que lo escribirías anidado, de afuera hacia adentro. Y el collect(groupingBy(c -> c, counting())) -agrupar por identidad y contar- es literalmente lo que hace Counter, así que desaparece: no se traduce, se reemplaza por la estructura de datos que existía para eso.

Notá que la versión Python no tiene un for explícito y sin embargo tampoco tiene un pipeline. Ese es el punto general: **casi todo pipeline de streams tiene una comprehension equivalente más corta**. Y cuando no la tiene -cuando la lógica es lo bastante compleja como para que la comprehension quede ilegible- el idioma correcto en Python es un for explícito, no una traducción forzada. Python no considera que el for sea una derrota; encadenar cinco operaciones funcionales para evitarlo, en cambio, sí es un java-ismo.

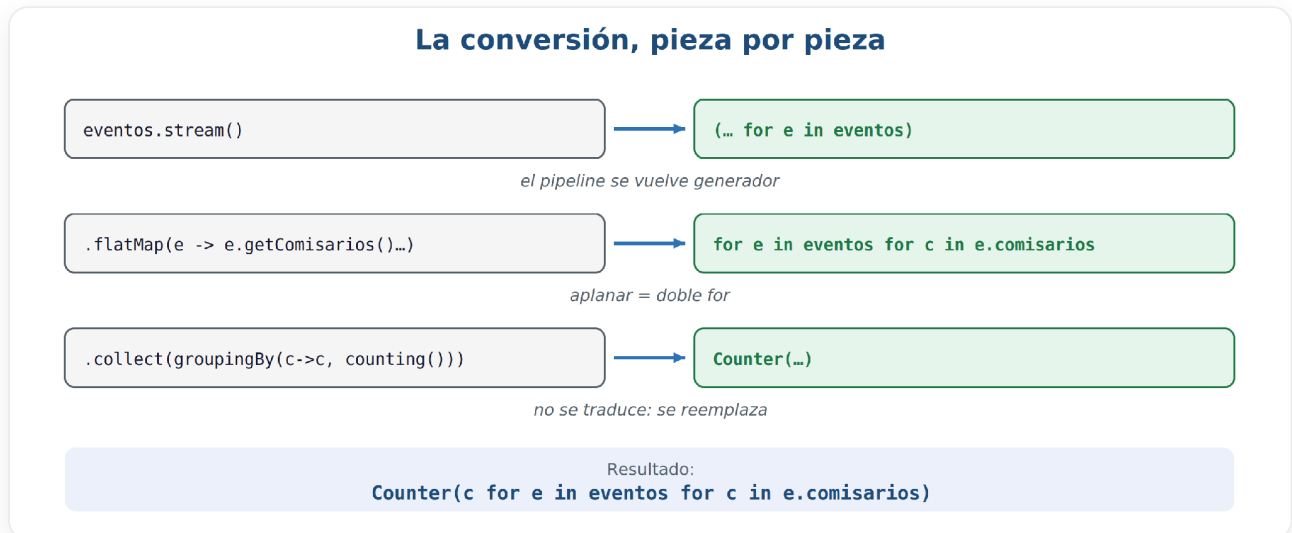


Figura 6 · La conversión pieza por pieza: el flatMap se vuelve doble for, y el collect no se traduce - se reemplaza por Counter.

Regla mental de conversión

Java Stream	Python
.filter(x -> cond)	[x for x in xs if cond]
.map(x -> f(x))	[f(x) for x in xs]
.flatMap(...)	doble for en la comprehension
.collect(groupingBy(...))	Counter / defaultdict(list)
.mapToDouble(...).sum()	sum(f(x) for x in xs)
.anyMatch(...) / .allMatch(...)	any(...) / all(...)
.sorted(comparing(...))	sorted(xs, key=...)
.distinct()	set(xs) o dict.fromkeys(xs)
.reduce(...)	functools.reduce(...) o sum/min/max
.limit(n)	itertools.islice(xs, n)
.count()	len(xs) o sum(1 for _ in xs)

4.2 · Sobrecarga de constructores

Java resuelve las variantes de construcción con múltiples firmas y deja que el compilador elija. Python no tiene sobrecarga: si escribís dos `__init__`, el segundo pisa al primero en silencio y el primero deja de existir. No hay error, no hay advertencia.

El idioma correcto es el **constructor alternativo** con `@classmethod`: un solo `__init__` que recibe lo esencial, y métodos de clase con nombre descriptivo para cada forma de construcción.

PYTHON

```
class Figura:

    def __init__(self, lados: list[Lado]) -> None:
        self._lados = lados

    @classmethod
    def vacia(cls) -> "Figura":
        return cls([])

    @classmethod
    def desde_medidas(cls, *medidas: float) -> "Figura":
        return cls([Lado(m) for m in medidas])
```

Fijate que esto es una mejora de legibilidad, no una concesión. En Java, `new Figura()` y `new Figura(3.0, 4.0, 5.0)` obligan al lector a mirar las firmas para saber cuál es cuál. En Python, `Figura.vacia()` y `Figura.desde_medidas(3, 4, 5)` se explican solos. El `cls` en lugar del nombre de la clase, además, hace que las subclases hereden los constructores alternativos y devuelvan instancias del tipo correcto - algo que en Java requiere trabajo extra.

4.3 · El punto y coma aplastado

Este es el reflejo más visible: escribir Python como si fuera Java al que se le borraron las llaves.

INCORRECTO - NO ES PYTHON

```
class Empleado: def __init__(self): self._rol_docente = None;
self._rol_investigador = None; self._rol_admin = None
```

PYTHON

```
class Empleado:

    def __init__(self) -> None:
        self._rol_docente: RolDocente | None = None
        self._rol_investigador: RolInvestigador | None = None
        self._rol_admin: RolAdmin | None = None

    def como_docente(self) -> RolDocente:
        if self._rol_docente is None:
            self._rol_docente = RolDocente()
        return self._rol_docente
```

La indentación **es** la sintaxis. El ; existe en Python -es legal- pero está proscripto por convención, y PEP 8 lo dice explícitamente. Enseñar por contraejemplo su uso, en un manual que en el cuerpo usa type hints y properties impecables, es una incoherencia que el estudiante copia sin pensar, porque asume que si está en el material está bien.

El caso de arriba tiene además un segundo problema, más de fondo: la versión aplastada esconde que `como_docente()` es donde vive el patrón. La composición de roles del Ejercicio 6 -el *overlapping*- se entiende leyendo ese método, no la lista de asignaciones. Cuando aplastás el código, lo que se pierde no es solo el estilo: es la estructura que hacía visible el diseño.

4.4 • Los tipos

El residuo más común y más fácil de pasar por alto: String donde va str. Es menor y no rompe nada, pero delata el origen del texto y, en un manual, le enseña al estudiante un tipo que no existe.

Java	Python
String	str
int / Integer	int
double / float	float
boolean / Boolean	bool
List<T>	list[T]
Map<K,V>	dict[K,V]
Set<T>	set[T]
null	None
Object	object
Optional<T>	T None
char	str (de longitud 1)
long	int (sin límite)
BigDecimal	decimal.Decimal
void	None (como tipo de retorno)

Dos filas merecen atención. El long de Java tiene 64 bits y desborda; el int de Python tiene precisión arbitraria y no desborda nunca - un factorial de 100 se calcula sin trucos. Y el char no existe: un carácter es un str de longitud uno, lo que significa que iterar un string te da strings, no otra cosa.

5 Trampas que Java no te enseñó a ver

Cuatro errores que en Java simplemente no pueden pasar

Este es el capítulo más urgente en la práctica. Son errores que Java te *impedía* cometer - así que tu intuición de programador Java no los detecta. No los vas a ver venir. Y dos de ellos producen bugs que se manifiestan lejos, en el tiempo y en el archivo, del lugar donde los escribiste.

① Default mutable compartido

El clásico absoluto, el que todo programador Python pisa una vez y no olvida nunca. El valor por defecto de un parámetro se evalúa **una sola vez, cuando se define la función** - no en cada llamada. La lista vacía que escribiste como default es *una* lista, creada al importar el módulo, y todas las instancias la comparten.

PYTHON

\# X TODAS las instancias comparten la MISMA lista

```
def __init__(self, lados: list[Lado] = []):  
    self._lados = lados  
    \# ✓ Correcto  
  
def __init__(self, lados: list[Lado] | None = None):  
    self._lados = lados if lados is not None else []
```

El síntoma es desconcertante: creás un Polígono nuevo, le agregás un lado, y aparece con los lados del polígono anterior. Como el defecto está en la *definición* y no en el uso, el bug se manifiesta a distancia y la sospecha nunca cae sobre la línea correcta.

CONSOLA - EL SÍNTOMA

```
>>> p1 = Poligono()
>>> p1.agregar_lado(Lado(3.0))
>>> p2 = Poligono() # "nuevo", recién construido
>>> p2.nro_lados
1 # ← ya nace con el lado de p1
>>> p1._lados is p2._lados
True # es literalmente la misma lista
```

La última línea es la que conviene mostrar en clase: `is` compara identidad, no contenido, y ese `True` dice que no hay dos listas parecidas sino una sola compartida. Es la evidencia directa de que el default se evaluó una única vez.

En Java esto no puede pasar porque el equivalente `-this(new ArrayList<>())-` ejecuta el `new` en cada llamada. Por eso tu intuición no tiene el reflejo. La regla mecánica es simple y no admite excepciones: **ningún default mutable, nunca**. Ni listas, ni diccionarios, ni sets, ni objetos. Siempre `None` y la construcción adentro.

El default se evalúa una sola vez

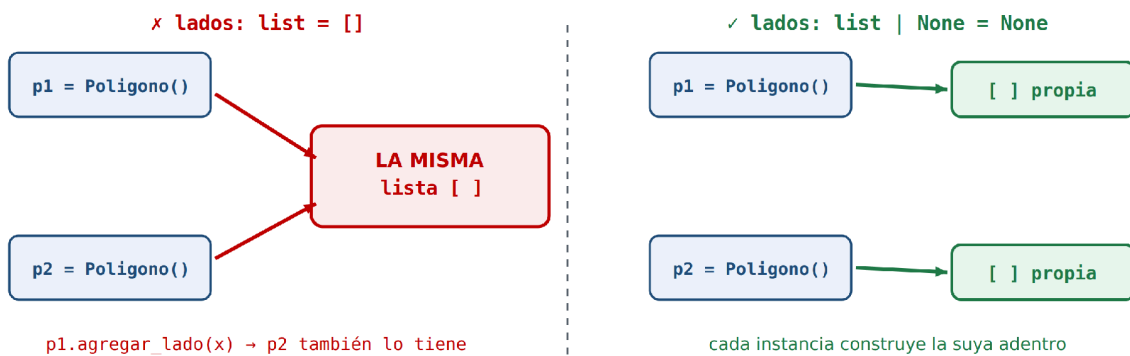


Figura 7 · El default se evalúa al definir la función: a la izquierda, ambas instancias comparten la misma lista; a la derecha, cada una construye la suya.

② Atributo de clase mutable - el static accidental

En Java, la diferencia entre un campo de instancia y un campo estático es una palabra clave visible que ninguna revisión pasa por alto. En Python la diferencia es la **indentación y el self** - dos cosas que el ojo entrenado en Java no lee como significativas.

PYTHON

```
class Poligono:
    lados: list[Lado] = [] # X es static: compartido por TODAS

    def __init__(self) -> None:
        self._lados: list[Lado] = [] # ✓ es de instancia
```

Lo insidioso es que la línea de arriba *parece* una declaración de campo al estilo Java -tipo, nombre, valor inicial- y es exactamente donde un programador Java esperaría declararlo. Pero en Python eso crea un atributo **de la clase**, no de la instancia: hay una sola lista para todos los polígonos del sistema.

La confusión se agrava porque con un `@dataclass` esa misma sintaxis *sí* declara campos de instancia - el decorador la reinterpreta. Y si intentás poner una lista como default en una dataclass, Python te tira un error explícito y te obliga a usar `field(default_factory=list)`. Es de los pocos lugares donde el lenguaje decidió protegerte activamente de esta trampa, precisamente porque es muy común.

③ `super().__init__()` olvidado

Java llama al constructor del padre automáticamente si no lo hacés vos: inserta un `super()` invisible al principio de tu constructor. **Python no**. Si no escribís `super().__init__()`, el constructor del padre simplemente no corre.

El resultado es un objeto a medio construir. En el ejemplo del Ejercicio 1, si `Poligono.__init__` se olvida del `super().__init__()`, todo lo que `Figura` iba a inicializar no existe - y el error aparece después, en el primer método que intente leer alguno de esos atributos, con un `AttributeError` que apunta a un lugar que no tiene nada que ver.

La regla práctica: **si tu clase hereda y define `__init__`**, la primera línea es `super().__init__(...)`. Sin excepciones y sin pensarlo. En jerarquías con herencia múltiple la llamada es además la que hace funcionar el MRO, así que saltarla rompe cosas más sutiles todavía.

④ Los type hints no validan nada

Esta es la trampa más peligrosa del capítulo, porque no produce un bug: produce **confianza infundada**.

`def agregar_lado(self, lado: Lado)` acepta un `str` sin chistar. Acepta `None`, acepta un `int`, acepta cualquier cosa. Los hints son documentación para el lector y metadatos para las herramientas; el intérprete los guarda en `__annotations__` y sigue de largo. No hay ninguna verificación en runtime, nunca.

El problema es que un código Python lleno de type hints *se parece* mucho a Java. Ves `def lados_esperados(self) -> int:` y una parte de tu cerebro registra «el compilador me cubre». No te cubre nadie. Si querés esa red, hay que instalarla:

SHELL

```
# En CI, no opcional:  
mypy --strict src/
```

Sin mypy corriendo en CI, un manual o un proyecto lleno de type hints es **peor** que uno sin hints, porque genera la sensación de tener garantías que no existen. Con mypy, en cambio, los hints se vuelven algo bastante parecido a un compilador - y encima uno más expresivo que el de Java en varios aspectos, porque puede expresar `T | None`, tipos literales, protocolos estructurales y genéricos variantes sin borrado de tipos.

El resumen honesto: Python te da un sistema de tipos tan bueno como el de Java, pero *opcional*. Y opcional significa que la decisión de tenerlo es tuya, no del lenguaje.